

TemaFinale26 – Sprint 1: Analisi del Problema

Scopo e goal dello Sprint 1

Questo documento sviluppa l'**analisi del problema** a partire dai requisiti già formalizzati ([TemaFinale26 – Analisi dei Requisiti](#)).

Goal dello Sprint 1. Produrre un **modello logico** del cargoservice — **struttura** (attori), **interazioni** (messaggi/eventi) e **comportamento** (FSM) — sufficiente a guidare la fase di Development, **motivando** la tecnologia adottata (qak/QActor) e individuando il **core business** da realizzare per primo.

Non disponendo del PicoW fisico, i dispositivi sono realizzati in modo **simulato** (Python/Java) con gli **stessi contratti** di un'eventuale realizzazione hardware: a livello logico conta l'*informazione* scambiata, non il dispositivo.

Perche' qak (QActor): motivazione

La scelta del modello a **attori** (qak/QActor) non e' arbitraria: discende dalla natura del cargoservice, che e' **al tempo stesso proattivo e reattivo**.

- **Proattivo:** accettata una richiesta, *guida* una sessione di carico in piu' passi (riserva slot → attende il container → IOPort→slot5→slot riservato → HOME), mantenendo uno **stato** tra un passo e l'altro.
- **Reattivo:** deve rispondere a *eventi asincroni e non prevedibili* — la pressione del pushbutton, l'arrivo del container (sonar), la fine della marcatura, un guasto del sonar, lo scadere del timeout di 30s.

Questa combinazione «macchina a stati + reazione a messaggi/eventi + temporizzazione» e' esattamente cio' che il modello QActor rende esplicito e di prima classe:

Esigenza del problema	Costrutto QActor
comportamento a fasi con memoria	State / Transition + variabili di stato
richiesta con risposta (pushbutton, robot)	whenRequest/replyTo ; request/whenReply
notifiche asincrone (sonar, marker)	whenEvent (publish/subscribe)
guasto che interrompe da qualunque stato	whenInterruptEvent (sonarFail → Out of service)
timeout dei 30 s	whenTime
disaccoppiamento da dispositivi simulati/reali	scambio di messaggi : stesso contratto per mock e hardware

In un'implementazione *monolitica* (un thread con cicli e flag) stato, attese, timeout e interruzioni andrebbero gestiti a mano e si confonderebbero tra loro; QActor li **separa** e li rende verificabili, e

disaccoppia il servizio dai dispositivi (sostituire il sonar simulato col PicoW reale non cambia la logica). Per questo qak, soltanto *anticipato* nei requisiti, viene qui **adottato e motivato**.

Architettura logica

Componente	Tipo	Ruolo
cargoservice	da costruire	FSM principale: richiesta, riserva slot, orchestra il cargorobot, aggiorna display/LED
cargorobot (DDR)	esterno (dato)	raggiunge una posizione con moverobot (X,Y,T); <i>quale</i> software del DDR lo realizzi (POJO RobotObj26 / RobotService26 / robotsmart) e' scelta di Progetto
iosensor	da costruire	filtra il sonar: emette containerAtIOPort e sonarFail
pushbutton / display / marker / LED	simulati	input richiesta / output stato / marcatura / lampeggio

Realizzazione dei dispositivi senza hardware

Dispositivo	Realizzazione per la demo
Sonar IOPort	mockPicowRadar.py → sonardata(D) su MQTT (identico al PicoW reale)
Pushbutton	messaggio loadrequest inviato da un client simulato (caller)
Display	output a video / log (show(...))
Marker	attesa simulata + segnale markingDone
LED	log a video del lampeggio (ledBlink)

Modello delle interazioni

```
//---- pushbutton -> cargoservice (richiesta con risposta)
Request loadrequest : loadrequest()
Reply  answer      : answer(ESP) for loadrequest "RESP = retrylater | reject | reserved(SLOT)"

//---- sonar (via iosensor) -> cargoservice
Event  containerAtIOPort : containerAtIOPort() "D<DFREE/2 per ~3s"
Event  sonarFail        : sonarFail()         "D>DFREE per ≥3s"
Event  sonarOk          : sonarOk()           "D≤DFREE"

//---- marker -> cargoservice
Event  markingDone      : markingDone()

//---- cargoservice -> cargorobot (contratto di robotsmart26)
Request moverobot       : moverobot(TARGETX, TARGETY, STEPTIME)
Reply  moverobotdone    : moverobotok(ARG)           for moverobot
Reply  moverobotfailed  : moverobotfailed(PLANDONE, PLANTODO) for moverobot

//---- display e LED: oggetti embedded (withobj), non messaggi qak
//  display.show(WHAT) con WHAT = reserved(SLOT)|retrylater|reject|serviceworking|outofservice|holdstate
//  led.blink(true|false)
```

Perche' questi tipi di interazione (non sono imposti dai requisiti: si scelgono qui, motivandoli):

- loadrequest → **request/reply**: il customer attende una *risposta* (reserved | retrylater | reject), quindi e' un'interazione richiesta–risposta;
- sonar e marker → **evento** (publish/subscribe): sono *sorgenti di notifiche* asincrone, emesse quando una condizione si verifica, senza attendere risposta. (E' una scelta: in alternativa potrebbero inviare *dispatch* diretti; si preferisce l'evento per disaccoppiare la sorgente dal servizio e permettere piu' osservatori.)
- moverobot → **request/reply**: il cargoservice deve sapere *quando* il robot ha raggiunto la posizione (moverobotok) per procedere al passo successivo; e' il contratto gia' offerto dal DDR.

Il **trasporto** concreto (TCP/CoAP/MQTT) **non** e' fissato qui: e' una decisione della fase di Progetto. A livello logico contano il *tipo* di interazione e l'informazione scambiata.

Core business e strategia incrementale

Concordato col committente (Sprint 0): il **core business** e' **portare un container, accettata la richiesta, dall'IOPort allo slot riservato passando per la marcatura in slot5**. Coerentemente, lo sviluppo procede in modo **incrementale**:

Iterazione	Cosa realizza
Prototipo 1 (core)	flusso nominale: richiesta valida → riserva slot → container all'IOPort → IOPort→slot5→slot riservato → HOME. Display/LED come log; <i>Out of service</i> e barcode ignorati (confermato dal committente).
Iterazione 2	rifiuti (hold pieno → reject; IOPort occupato → retrylater) e timeout dei 30s.
Iterazione 3	<i>Out of service</i> (guasto sonar) e ripristino; display come pagina web .

Il modello qak qui sotto e' presentato **completo** (include gia' Out-of-service e timeout) per dare una visione d'insieme; la **demo significativa** riguarda pero' il Prototipo 1 (vedi *Piano di test*).

Comportamento del cargoservice (modello qak)

Il comportamento e' espresso direttamente in qak (QActor):

```
QActor cargoservice context ctxcargo
  withobj hold using "domain.Hold(...)" //slot1..5, posizioni, mappa
  withobj display using "devices.DisplaySim()" //display simulato (output a video)
  withobj led using "devices.LedSim()" { //LED simulato (lampeggio a video)
[# var Slot = ""
  var OutOfService = false
  val StepTime = 345
#]
State s0 initial {
  println("$name | starts") color blue
  [# display.show("serviceworking") #]
}
```

Goto available

```
State available {                                //Service working
  println("$name | available") color yellow
}
```

```
Transition t0
  whenRequest loadrequest    -> evalRequest
  whenInterruptEvent sonarFail -> goOutOfService
```

```
State evalRequest {
  onMsg( loadrequest : loadrequest() ){
    [# Slot = "" #]
    if [# OutOfService || hold.ioportOccupied() #] {
      replyTo loadrequest with answer : answer(retrylater)
      [# display.show("retrylater") #]
    }
    if [# !(OutOfService || hold.ioportOccupied()) && hold.isFull() #] {
      replyTo loadrequest with answer : answer(reject)
      [# display.show("reject") #]
    }
    if [# !(OutOfService || hold.ioportOccupied()) && !hold.isFull() #] {
      [# Slot = hold.reserveFreeSlot() #]
      replyTo loadrequest with answer : answer(reserved($Slot))
      [# display.show("reserved " + Slot); led.blink(true) #] //engaged
    }
  }
}
```

Goto waitContainer if [# Slot != "" #] else available

```
State waitContainer {                            //engaged: attende il container entro 30s
  println("$name | engaged, attendo container (max 30s)") color magenta
}
```

```
Transition t0
  whenEvent containerAtIOPort -> goToIOPort
  whenTime 30000               -> disengage      //timeout 30s
  whenInterruptEvent sonarFail -> goOutOfService
```

```
State goToIOPort {                               //il robot raggiunge l'IOPort (presa astratta)
  [# val IX = hold.ioportX(); val IY = hold.ioportY() #]
  request cargorobot -m moverobot : moverobot($IX, $IY, $StepTime) //HOME -> IOPort
}
```

```
Transition t0
  whenReply moverobotdone    -> carryToSlot5
  whenInterruptEvent sonarFail -> goOutOfService
```

```
State carryToSlot5 {
  [# val SX = hold.slotX("slot5"); val SY = hold.slotY("slot5") #]
  request cargorobot -m moverobot : moverobot($SX, $SY, $StepTime) //IOPort -> slot5
}
```

```
Transition t0
  whenReply moverobotdone    -> marking
  whenInterruptEvent sonarFail -> goOutOfService
```

```
State marking {                                  //il marker etichetta il container
  println("$name | marcatura in slot5 ...") color cyan
}
```

```
Transition t0
  whenEvent markingDone      -> carryToReserved
  whenInterruptEvent sonarFail -> goOutOfService
```

```
State carryToReserved {
  [# val SX = hold.slotX(Slot); val SY = hold.slotY(Slot) #]
  request cargorobot -m moverobot : moverobot($SX, $SY, $StepTime) //slot5 -> slot riservato
}
```

```

Transition t0
  whenReply moverobotdone    -> goHome
  whenInterruptEvent sonarFail -> goOutOfService

State goHome {
  [# hold.confirmStored(Slot)
    val HX = hold.homeX(); val HY = hold.homeY() #] //posizione HOME dalla mappa del DDR
  request cargorobot -m moverobot : moverobot($HX, $HY, $StepTime) //ritorno a HOME
}
Transition t0
  whenReply moverobotdone    -> doneRequest
  whenInterruptEvent sonarFail -> goOutOfService

State doneRequest {
  [# led.blink(false) #]          //disengaged
  [# display.show("holdstate") #]
}
Goto available

State disengage {
  //timeout: il container non e' arrivato
  println("$name | timeout 30s: disengaged") color red
  [# hold.freeSlot(Slot); led.blink(false) #]
}
Goto available

State goOutOfService {
  //sonarFail
  [# OutOfService = true; led.blink(false); display.show("outofservice") #]
  println("$name | OUT OF SERVICE (sonar)") color red
}
Transition t0
  whenEvent sonarOk -> backInService

State backInService {
  [# OutOfService = false #]
}
Goto available
}

```

Il **iosensor** e' un QActor di supporto che si iscrive al sonar (mock MQTT o sonar di WEnv) e applica i filtri temporali, emettendo **containerAtIOPort** ($D < D_{FREE}/2$ per $\sim 3s$), **sonarFail** ($D > D_{FREE}$ per $\geq 3s$) e **sonarOk** ($D \leq D_{FREE}$). Così' il cargoservice resta indipendente dalla sorgente fisica del sonar.

Flusso nominale (richiesta accettata)

```

customer      cargoservice      cargorobot      marker
|loadrequest() |                |                |
|----->| (non OutOfService, IOPort libero, hold non pieno)
|         | reserve slot2; led.blink(true); display.show(reserved slot2)
|answer reserved(slot2)         |                |
|<-----|                |                |
|         | (engaged, attende container all'IOPort entro 30s)
| (sonar:  $D < D_{FREE}/2$  per 3s) -> containerAtIOPort() |
|         |---- moverobot(IOPort) ----->|         HOME -> IOPort
|         |<--- moverobotok -----|
|         |---- moverobot(slot5) ----->|         IOPort -> slot5
|         |<--- moverobotok -----|
|         | (marcatura)                | markingDone()

```

<----- -----	
--- moverobot(slot2) ----->	slot5 -> slot riservato
<--- moverobotok -----	
--- moverobot(HOME) ----->	ritorno a HOME
<--- moverobotok -----	
LED off; show holdstate; -> available	

Decisioni sulle problematiche

P1 — Dispositivi simulati. sonar = mockPicowRadar.py (MQTT, identico al PicoW reale); pushbutton = messaggio loadrequest da un caller; display/LED = log a video; marker = attesa + markingDone. I contratti restano invariati: sostituendo il mock col PicoW reale, il cargoservice non cambia.

P2 — Deposito astratto. Il container si considera spostato quando il cargorobot **raggiunge** la posizione (slot5, poi lo slot) con moverobot: nessun attuatore di presa richiesto.

P3 — Out of service. sonarFail e' gestito come transizione da qualunque stato verso goOutOfService (display "Out of service", LED off, richieste rifiutate con retrylater); il servizio riprende su sonarOk.

Piano di test dello Sprint1

Il cargoservice mantiene lo stato della stiva, che evolve in conseguenza dei comandi: il test e' automatizzabile confrontando lo stato finale con quello atteso.

1. **Carico nominale (PASS):** stiva con slot liberi, loadrequest, container all'IOPort entro 30s. Al termine si verifica:

```
slot riservato .occupied == true
posizione robot == HOME
LED == off && display == holdstate
stato cargoservice == available
```

2. **Hold pieno (PASS se rifiutato):** slot1..4 occupati → show(reject), stiva invariata.
3. **IOPort occupato / Out of service (PASS se retrylater):** richiesta mentre occupato o in OutOfService → show(retrylater).
4. **Timeout (PASS se disengaged):** richiesta accettata ma container non arriva entro 30s → slot liberato, LED off, ritorno ad available.
5. **Out of service (PASS):** sonar D>DFREE per 3s → show(outofservice); al ritorno D≤DFREE → available.

Il caso 1 e' il **Test Plan significativo** per la demo (richiesto dalla valutazione): copre l'intero ciclo e termina con assert PASS/FAIL integrabili in JUnit o nel log di collaudo.



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer